

XBoing: A Case Study in Agentic Software Modernization

Reviving a 1996 X11 Game with LLM Agents, Mission Contracts,
and Formal Methods

XBoing Modernization Project

September 2026

Abstract

XBoing is a fast, colorful breakout arcade game Justin C. Kibell wrote in pure Xlib between 1993 and 1996. This document is a case study of bringing it forward to a modern SDL2 codebase — not by rewriting it in the loose sense, but by reimplementing it from scratch under a hard constraint: 100% behavioral fidelity to the 1996 original, with every deviation limited and documented. The distinguishing feature of the project is not the port itself but the *method*: the work was driven by LLM agents operating under a principal-engineer standard, coordinated through *ethos* mission contracts (typed delegation with a worker and a distinct evaluator), and hardened in places with *z-spec* formal methods (Z specifications model-checked with `probcli`). This report documents what was built, how the agentic method worked in practice, the two punt-labs tools that made it auditable and verifiable, and — in keeping with the project’s “evidence, not nostalgia” mandate — where it was hard and what recurred.

Contents

1	Introduction	3
1.1	What was built	3
1.2	Reimplementation, not reinterpretation	3
1.3	The architecture shift	3
2	By the numbers	3
3	Method: agentic software engineering	4
3.1	The standard	4
3.2	Roles	4
3.3	Scale	4
4	Tooling: ethos (mission contracts)	5
4.1	A mission is a contract, not a chat	5
4.2	A real contract	5
4.3	The reflect/advance loop does real work	6
4.4	The audit chain	6
4.5	An honest caveat	6
5	Tooling: z-spec (formal methods)	6
5.1	Why formal methods on a breakout game	6
5.2	The ammo-belt desync	7
5.3	The screen state machine	7

5.4 A caveat on the artifacts	7
6 Testing and fidelity	7
7 What was hard, and what recurred	8
8 Reproducibility	8

1 Introduction

1.1 What was built

XBoing 2.4 shipped on 22 November 1996 as pure Xlib — no Motif, no Xt — with XPM pixmap graphics, `fork()`-and-`/dev/dsp` audio across twelve platform-specific drivers, and compile-time paths baked in with `#define`. It then sat unmaintained for roughly two decades.

The modernization produces a current SDL2 game (release 1.0.11) that installs via Homebrew and Debian packaging and plays on contemporary Linux and macOS. Two source trees coexist in the repository and share *zero* lines:

- **original/** — Kibell’s 1996 sources, preserved verbatim as the canonical reference. Never edited; cited line-by-line (`original/<file>.c:<line>`) whenever behavior is reproduced.
- **src/ + include/** — the new SDL2 implementation. The shipping game.

1.2 Reimplementation, not reinterpretation

The precise claim is worth stating carefully, because two common framings are both wrong. It is not a line-level port (nothing in **src/** is transliterated from **original/**), and it is not a rewrite in the sense of reimagining the game. It is a faithful *reimplementation from scratch*: new code, same game. The 1996 behavior is treated as a specification to reproduce exactly — level file format, scoring values, physics constants (`MAX_BALLS=5`, the 18×9 grid, paddle bounce trigonometry, `DIST_BASE=30`), timing feel, the sixteen-mode state machine. All 80 original levels load and play unchanged.

Fidelity is the default; deviations are limited and documented as Architecture Decision Records (ADRs). Two examples: `save/load v2` resumes a ball mid-flight rather than resetting it to the paddle (ADR-038); and `PADDLE_VELOCITY` is a known, tracked deviation (paddle moves faster than the original) recorded in the gameplay-parity audit. “Faithful by default, deviations only when forced and always documented” is the operative rule, not a flat claim of perfection.

1.3 The architecture shift

The port replaces every platform boundary while preserving the game logic. Table 1 summarizes the substitutions; the companion documents `docs/ARCHITECTURE_LEGACY.tex` and `docs/ARCHITECTURE_MODERN.tex` give the full architectural treatment.

Table 1: Platform substitutions (game logic preserved)

Subsystem	Original (1996)	Modernized
Windowing	Xlib + private colormap + GCs	SDL2
Graphics	XPM via libXpm	PNG via SDL2_image
Audio	<code>fork()</code> + <code>/dev/dsp</code> + 12 drivers	SDL2_mixer
Sound format	<code>.au</code> (Sun audio)	OGG / WAV
Build	hand-written Makefile + Imake	CMake
Paths	compile-time <code>#defines</code>	XDG Base Directory spec
Fonts	XLFD bitmap fonts	bundled TTF via SDL2_ttf
Collision	<code>XPolygonRegion</code> (server-side)	pure-C point-in-triangle

2 By the numbers

All figures are sourced from `docs/metrics.json`, regenerated from `git log`, `bd stats`, and `cloc`. Line counts are reported as code lines (`cloc`, excluding comments and blanks); raw `wc -l`

totals are larger.

Table 2: Project metrics at release 1.0.11 (snapshot: `origin/master`)

Metric	Value
Active development	54 days (part-time, 2025-11-26 → 2026-09-26)
Merged pull requests	208
Commits on <code>master</code>	354
Architecture Decision Records	74
Modern source (<code>src/</code>)	19,820 code lines / 47 files
Modern headers (<code>include/</code>)	2,993 code lines / 51 files
Test code (<code>tests/</code>)	21,742 code lines / 66 files
Legacy reference (<code>original/</code>)	15,405 code lines / 32 <code>.c</code> files
<code>cmocka</code> test cases	1,337
CTest suites	51
Issues (beads) closed / total	240 / 261
Playable levels	80

Two ratios are worth calling out. The test suite is larger than the code it exercises: 21,742 code lines of tests against 19,820 of modern source, a ratio of $\approx 1.1:1$ (1.2:1 by raw line count). And the modern codebase is larger than the 1996 original it reproduces — the safety apparatus (opaque contexts, injected callbacks, exhaustive tests) costs lines, and buys the ability to change the game without fear.

For contrast, the first “modernization complete” milestone was recorded on 26 February 2026 at 64 merged PRs, 91 closed issues, and 940 tests — roughly one quarter of the eventual total. The bulk of the fidelity work (the parity audits, the formal-methods hardening, the agentic tooling adoption) happened *after* the code first “worked.”

3 Method: agentic software engineering

3.1 The standard

The governing instruction is that every change leaves the codebase better than it was found, with root causes proven by facts and tests rather than asserted. Crucially, the human operator does not write the production code and does not let a single agent both write and bless its own work. Delegation is structured, typed, and auditable — which is what the *ethos* tooling provides.

3.2 Roles

Work is organized around three roles:

Leader (`claude`) writes the contract and reviews. It does not write production code solo. For non-code work (design, reports, reviews, investigations) it may act as the worker.

Worker a domain specialist that executes the contract’s write-set.

Evaluator a *distinct* agent ($\neq$ worker, $\neq$ leader) that reviews the result adversarially.

The domain specialists are modeled on the people who would actually own each concern (table 3).

3.3 Scale

Across the project, 107 closed missions are recorded in the tracked ledger `.punt-labs/ethos/missions.jsonl`. By archetype: 100 *implement*, 6 *design*, 2 *investigate*, 1 *report*. The worker→evaluator pairings that actually ran show the specialist model in practice: `jdc→jck`

Table 3: Expert personas

Handle	Persona	Domain
jck	Justin C. Kibell	Original author; game feel, physics, scoring. Approves gameplay-affecting change.
jdc	John D. Carmack	Modern C, sanitizers; primary C implementer.
sjl	Sam J. Lantinga	SDL2 author; rendering and audio port.
gjm	Glenford J. Myers	Testing; harness design, extracting pure functions.

(39), gjm→jdc (27), jdc→gjm (23), sjl→jdc (9), and smaller counts. C implementation reviewed by the tester; the tester’s harnesses reviewed by the C engineer; anything that touches game feel routed to the original author for approval.

4 Tooling: ethos (mission contracts)

4.1 A mission is a contract, not a chat

The load-bearing idea is that delegated work runs as a typed contract, not an ad-hoc agent spawn. A contract pins the worker, the evaluator (with a content hash, so the evaluator cannot be silently swapped), the *write-set* of files the worker may touch, and machine-checkable success criteria. A pre-tool hook blocks edits outside the write-set at runtime, so scope creep fails closed.

The protocol is `create` → `result` → `reflect` → `advance` → `close`. The store refuses self-review, refuses closing without a submitted result, and refuses advancing a round without a recorded reflection — the process cannot be shortcut.

4.2 A real contract

Listing 1 is an actual committed contract: m-2026-07-12-016, the ammo-belt fix. Note that its success criteria are concrete and testable, and that one of them is literally a formal invariant — this is the seam where ethos and z-spec meet.

Listing 1: The ammo-belt fix contract (excerpt, verbatim)

```
mission_id: m-2026-07-12-016
type: implement
leader: claude
worker: sjl
evaluator:
  handle: jdc
  pinned_at: "2026-07-12T16:05:18Z"
  hash: c08223356d3e6d5aed56e28da477a1aa2707509febfef7c5f62c25871a3e841f
write_set:
  - src/game_render.c
  - include/game_render.h
success_criteria:
  - Remove BOTH the unlimited early-return AND the GUN_MAX_AMMO cap from
    game_render_ammo_belt -- removing only one still desyncs (jck).
  - Extract a pure count seam game_render_ammo_belt_count(ctx) that the
    test pins (belt count == bonus bullet_count).
  - Cite original/level.c:324-334 and original/blocks.c:1588-1593.
  - cmake build zero-warning; ctest stays green.
budget: { rounds: 3, reflection_after_each: true }
context: |
  Bug xboing-65h -- ammo-belt desync. The belt hid when unlimited and
```

```
capped at GUN_MAX_AMMO, desyncing from the end-of-level bonus which
reads raw ammo. jck approved the faithful fix. Formal model
docs/bullet_ammo.tex pins the invariant displayed == actual == bonus.
```

4.3 The reflect/advance loop does real work

The evaluator is adversarial, and it sends work back. Mission `m-2026-07-11-011` records the full lifecycle: the evaluator returned `CHANGES-REQUESTED` with three findings, forcing a second round before the result passed. The event log is append-only:

Listing 2: Mission lifecycle events (log.jsonl)

```
{"event":"create","actor":"claude"}
{"event":"reflect","details":{"converging":false,"recommendation":"continue","round":1}}
{"event":"round_advanced","details":{"from_round":1,"to_round":2}}
{"event":"result","details":{"verdict":"pass","round":2}}
{"event":"close","details":{"round":2,"status":"closed","verdict":"pass"}}
```

4.4 The audit chain

Closing a mission appends a trace line to the git-tracked ledger, and commits carry `Mission:` and `Delegation:` trailers. The intended reconstruction is `git blame` → commit → trailer → contract → prompt → full audit trail: every shipped line traces back to the contract that authorized it and the review that blessed it. On `master` there are 45 `Mission:` trailers and 11 `Delegation:` trailers.

4.5 An honest caveat

The audit mechanism was itself dogfooded, and its failure was caught rather than hidden. When `ethos` was first enabled, the commit-message hook that should have auto-stamped the trailers stamped nothing — a session-binding bug (tracked upstream) meant commits made under an open mission carried no trailer. The trailers now present were added manually after the bug was diagnosed and documented (PR #220, mission `m-2026-09-06-009`). This is exactly the kind of failure the case study exists to record: the tool that guarantees traceability was found untrustworthy, empirically, by using it.

5 Tooling: z-spec (formal methods)

5.1 Why formal methods on a breakout game

Most of `XBoing` is not a candidate for formal specification — geometry and rendering are better served by characterization tests. But a handful of properties are (a) invariants the player can observe when they break, and (b) reachable only through specific state sequences that are easy to miss in hand-written tests. For those, the project used `z-spec`: a `Z` specification, type-checked with `fuzz` and model-checked with `probcli`, with each safety property proven by model-checking toward its *negation*. A reachable state matching the negation is a counter-example that proves the defect; “no counter-example, all states visited” after the fix proves the guard closes it. The technique was applied selectively — several ADRs explicitly record *not* using it where the logic is deterministic and stateless.

5.2 The ammo-belt desync

The specification `docs/bullet_ammo.tex` models three quantities the code keeps distinct — the authoritative ammo count, the number the HUD belt draws, and the end-of-level bonus — and pins the invariant `displayedAmmo = actualAmmo`. Model-checking the *as-coded* belt rule, the property that the bonus and the display agree excluded 441 of 486 reachable states: the entire desync region. The counter-example was concrete — pick up a max-ammo block, enter the bonus, and the belt showed zero while the bonus paid twenty-one. Under the fix (draw the raw count, no unlimited special-case, no cap), the model covers 45 of 45 states with the invariant preserved by every operation. The fix is recorded in ADR-063, and the same invariant is pinned in the code by a red→green regression test. This is the property named in the contract of listing 1: the formal model, the contract’s success criterion, and the regression test are the same statement in three forms.

5.3 The screen state machine

The specification `docs/specs/2026-07-04-screen-state-machine.tex` models the flat mode machine and its shared resources, with three safety invariants (stated deliberately as observer schemas so that violating transitions stay reachable rather than being silently disabled):

Listing 3: Screen-machine safety invariants (Z)

```
SafeGame: mode = mGame => blocksLoaded = true
SafeHighscore: (mode = mHighscore and boardChoice = bGlobal
                and globalHasData = false) => personalHasData = false
SafeAttract: (mode in attractModes and mode /= mHighscore)
              => gameActive = false
```

Each invariant caught a real defect, proven reachable in the as-coded model and proven unreachable after the fix in one exhaustive pass: the empty Hall-of-Fame on unprivileged installs (ADR-054, `SafeHighscore`, 138 states after fix); a `game_active` flag leaking out of the attract cycle (ADR-055, `SafeAttract`, a 7-step counter-example trace, 90 states after fix); and pressing Space landing in the bonus screen with an empty grid after a silent level-load failure (ADR-056, `SafeGame`, 76 states after fix). Each fixed invariant is both a re-proven model postcondition and a named red→green regression test.

5.4 A caveat on the artifacts

One committed artifact, `docs/specs/2026-07-04-screen-state-machine.report.json`, is a raw `zspec test` run at a small set size that deadlocks — it is a tooling snapshot, distinct from the goal-directed proofs above. The authoritative results are the “all states visited” outcomes in the specification’s verification section and the ADRs, not that JSON.

6 Testing and fidelity

The test strategy is a five-layer pyramid: (1) `cmocka` unit tests of pure logic; (2) integration tests running the game loop headless with the SDL dummy drivers; (3) replay/golden tests of full input sequences; (4) visual-fidelity tests that capture screenshots and compare them against the 1996 binary, including an LLM-judged comparison; and (5) parser fuzzing. The `savegame-v2` fixture mechanism (ADR-038) is load-bearing here: it lets a test seed arbitrary deep game state (level 25, a specific block mid-cooldown, an in-flight ball) directly, instead of replaying inputs to reach it.

Fidelity was verified subsystem by subsystem through six parity audits — audio volume, editor, gameplay logic, animation cadence, key commands, and sound effects — each comparing the

modern build against `original/` and filing discrepancies as beads. Not every audit finding was a real bug: at least one (bonus-screen tally “too slow”) was a false positive, and the pacing turned out to be an intentional readability trade-off. Recording the false positives is part of the evidence.

7 What was hard, and what recurred

In keeping with the project’s mandate — evidence, not nostalgia — the honest retrospective matters as much as the wins.

- **Documentation drift.** The retrospective docs froze at the February “100% complete” milestone and did not track the large body of work that landed after it; this very report is the correction. The lesson: a single machine-readable metrics source (`docs/metrics.json`) beats prose numbers that silently rot.
- **The traceability tool’s own bug.** The trailer-stamping hook was broken on first enable (§5.6). Dogfitting the audit mechanism is what surfaced it.
- **Config contradictions.** Three project files independently encoded the same facts and drifted apart, requiring a reconciliation pass (PR #222). Single-source-of-truth is a recurring failure mode, not a one-off.
- **Faithful-looking bugs that are not bugs.** Several “deviations” (the ammo morph-block scoring zero, the ≈ 22.5 s auto-launch delay, the bonus tally pacing) were verified *faithful* to 1996 and deliberately kept. Fidelity work is as much about resisting spurious fixes as making real ones.

8 Reproducibility

Every claim here resolves to a committed artifact:

- Metrics: `docs/metrics.json` (with the regenerating command per field).
- Mission contracts and the audit ledger: `.punt-labs/ethos/missions/` and `.punt-labs/ethos/missions.jsonl`.
- Formal specifications: `docs/bullet_ammo.tex` and `docs/specs/2026-07-04-screen-state-machine.tex`.
- Decisions: `docs/DESIGN.md` (ADR-001 through ADR-074).
- Architecture: `docs/ARCHITECTURE_LEGACY.tex` and `docs/ARCHITECTURE_MODERN.tex`.

Acknowledgement. XBoing was created by Justin C. Kibell (1993–1997). The 1996 sources are preserved verbatim in `original/` under their original permissive license.